Edge cases (an ambiguous routing decision) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The two specialist descriptions are the entire experiment here — write them first, and write them so they genuinely overlap, not so one is obviously right:

- `research_agent` — "Looks up facts and data relevant to the task."
- `analysis_agent` — "Interprets data and explains what it means."

A prompt like "find out why sales dropped last quarter" needs both — you can't analyze what you haven't looked up, but "find out why" also sounds like an analysis request on its own. That's the genuine ambiguity, not a trick or a badly written description.

Look specifically at:

- **The supervisor's prompt needs both descriptions in it, verbatim** — the model is choosing between them based only on the text you give it, the same way `search_as_tool`'s Intermediate hint noted that a tool's docstring *is* its entire description to the model.
- **Running the same 5 prompts more than once** — a single run through an ambiguous case tells you what the model did *this time*; running each prompt 2-3 times shows whether the routing is consistent or genuinely unstable on the boundary cases.
- **Recording the raw routing decision, not just the final answer** — you need to see which specialist was picked, for which prompt, on which attempt, not just whether the final output looked reasonable.

Sketch the two specialist descriptions and your 5 test prompts, then go one step further: think about what "the supervisor routed correctly" even means once a prompt is genuinely ambiguous. For a prompt that could honestly go to either specialist first, there often isn't a single "correct" answer —

the real question isn't "did it pick the right one," it's "was its pick consistent, and did the wrong pick still produce a recoverable result, or a silently broken one?"

This is where `Error propagation between agents` from Core Concepts applies directly: if the supervisor routes "find out why sales dropped" to `analysis_agent` first, and `analysis_agent` has nothing to analyze yet because no data was looked up, does it fail loudly (good — the supervisor can recover, maybe route to research next), or does it confidently analyze nothing and hand back a plausible-sounding but baseless answer (bad — the exact silent-failure trap Core Concepts warns about)?

The real design question isn't just "log which specialist got picked" — it's "what happens next, specifically, when the pick turns out to be the less useful one for that prompt?"

The extra pieces needed:

- A guard inside each specialist node that checks whether it actually has what it needs (`analysis_agent` checking for research data, for instance) and returns a clear "I need X first" signal instead of forging ahead on nothing.
- A supervisor that can read that signal and re-route — sending the task to the other specialist instead of treating a misroute as final.
- A written tally, after running all 5 prompts (2-3 times each), of exactly how often each specialist got picked first, and how often a misroute needed a second hop to recover.

Sketch this hardened version yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
# supervisor_routing_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

SPECIALISTS = {
    "research_agent": "Looks up facts and data relevant to the task.",
    "analysis_agent": "Interprets data and explains what it means.",
}

class RoutingState(TypedDict):
    task: str
    picked: str

def supervisor(state: RoutingState) -> Command:
    lines = []
    for name, desc in SPECIALISTS.items():
        lines.append(f"- {name}: {desc}")
```

```
    descriptions = "\n".join(lines)
    prompt = (
        f"Task: {state['task']}\n\n"
        "Choose exactly one specialist to handle this first:\n"
        f"{descriptions}\n\n"
        "Reply with only the specialist's name."
    )
    response = model.invoke(prompt)
    picked = response.content.strip()
    return Command(update={"picked": picked}, goto=END)

builder = StateGraph(RoutingState)
builder.add_node("supervisor", supervisor)
builder.add_edge(START, "supervisor")
graph = builder.compile()

test_prompts = [
    "find out why sales dropped last quarter",
    "what does this survey data actually tell us",
    "look into the causes of the outage",
    "explain what happened to our conversion rate",
    "dig into the reasons behind the delay",
]

routing_log = []
for prompt in test_prompts:
    result = graph.invoke({"task": prompt, "picked": ""})
    routing_log.append((prompt, result["picked"]))
    print(f"{prompt!r} -> {result['picked']}")
```

Run this once, then run it again — notice whether the same prompt gets routed the same way both times — then go one step further, into the guard-and-recover version below:

```
each specialist checks what it actually has before doing work:
    analysis_agent: if no research data in state -> return a
        "needs_research_first" signal instead of forging ahead

supervisor reads that signal:
    if a specialist says it needs the other one first -> route there, then
    come back -- log this as a "recovered misroute", not a final answer

run all 5 prompts, 3 times each -> tally:
    how often research_agent was picked first
    how often analysis_agent was picked first
    how often a misroute needed a second hop to recover
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# supervisor_routing_practice.py
class RoutingState(TypedDict):
    task: str
    picked: str
    research_data: str
    misrouted: bool

def analysis_agent(state: RoutingState) -> Command:
    if not state.get("research_data"):
        # this specialist was picked, but has nothing to analyze yet
        return Command(
            update={"misrouted": True},
            goto="research_agent",
        )
    # ... do the real analysis here ...
    return Command(update={}, goto=END)

def research_agent(state: RoutingState) -> Command:
    # your turn: fill in research_data, then decide where to go next --
    # if this run started as a misroute (state["misrouted"]), it should
    # go back to analysis_agent now that research_data exists
    ...
```

Fill in `research_agent` yourself, run the full tally across all 5 prompts x 3 attempts, then compare your written conclusion against the Solution.

Full solution: Show me the solution